

Adaptive Consistency: Enforcement, Completeness, and the Width Formalisms

A literature review and annotated bibliography, oriented toward the Lean mechanization in `Ste.BucketConsistency`

Project `thejeb`

July 24, 2026

Abstract

The repository’s `Ste.AdaptiveConsistency` module mechanized the *sufficiency* half of the adaptive-consistency theorem: a bucket network that is directionally consistent along an elimination order admits a backtrack-free solution. This review surveys the literature behind the *other* half — that adaptive consistency (bucket elimination with existential projection) *establishes* directional consistency while *preserving* the solution set, at cost exponential only in the induced width — and behind the identification of induced width with graph treewidth. It then maps the surveyed results onto the repository’s existing Lean modules (`Ste.Elimination`, `Ste.Projection`, `Ste.Treedecomp`, `Ste.GraphTreewidth`, `Ste.AdaptiveConsistency`) and onto the new module `Ste.BucketConsistency`, stating precisely which fragments are machine-checked and which remain open. Following the project rule — distrust results that are not machine proven — all claims about the mechanization are limited to statements that compile in CI; everything else is attributed to the cited literature.

1 The problem

A finite constraint network consists of variables $x_0, \dots, x_n$ with domains D_i and a set of relations (constraints), each over a subset of the variables (its *scope*). Three questions dominate the structural theory: (i) when does *local* consistency imply *global* solvability; (ii) can local consistency of the required strength be *enforced* by an algorithm that does not change the solution set; and (iii) what *graph parameter* of the constraint network governs the cost of that enforcement. The classical answers — directional $(w+1)$ -consistency suffices along an order of induced width w [6, 8]; adaptive consistency / bucket elimination enforces it in $O(n k^{w+1})$ time and space while preserving the solution set [4–6]; and induced width is graph treewidth [1, 3] — together constitute the tractability theory of bounded-width constraint solving, culminating in Grohe’s converse [10].

The repository proved (i) in Lean; this review is the survey component of the attack on (ii) and (iii). Section 2 reviews the classical results with an eye to what a mechanization must supply; Section 3 reviews the width formalisms; Section 4 audits the mechanization, including a gap the enforcement analysis surfaced in the repository’s own statement of directional consistency; Section 5 is the annotated bibliography.

2 Local consistency and backtrack-free search

2.1 From arc consistency to width

Mackworth systematized the local-consistency propagation algorithms (node, arc, and path consistency) as fixed-point computations on a network of relations [11]. Arc consistency alone does not imply solvability; the decisive insight is Freuder’s: what matters is the *relationship* between the level of local consistency and a structural parameter of the constraint graph. Freuder defined the *width* of a vertex ordering as the maximal number of earlier neighbours of any vertex, and the width of a graph as the minimum over orderings, and proved: if a network is strongly $(w+1)$ -consistent and its constraint graph has width w , then a solution can be assembled greedily along the ordering with no backtracking [8]. For trees ($w = 1$) arc consistency suffices — the case mechanized in `Ste.ConsistencyTree`. Freuder later generalized the statement to *backtrack-bounded* search: b -level consistency relative to orderings with bounded “width at level b ” bounds the depth of backtracking rather than eliminating it [9]; the k -tree connection implicit there — graphs of width w in Freuder’s sense are exactly partial w -trees — is the bridge to the treewidth literature (Section 3).

The subtlety that drives everything afterwards: enforcing $(w+1)$ -consistency by the naive closure algorithms *adds constraints*, which adds edges to the constraint graph, which can *increase its width*. Freuder’s theorem therefore does not by itself yield an algorithm: the hypothesis (consistency level) and the parameter (width) chase each other. Two escapes were found. Seidel’s “invasion” procedure processes the variables in one direction and records, for each frontier, the set of partial solutions — solving the network in time exponential in the maximal frontier size [13]. Dechter and Pearl’s *adaptive consistency* is the refined form: rather than enforcing a uniform consistency level everywhere, adapt the level per variable, exactly matching each variable’s set of earlier neighbours *in the graph as modified by the algorithm’s own recorded constraints* [6].

2.2 Adaptive consistency and bucket elimination

Fix an ordering $x_0 < x_1 < \dots < x_n$ and process variables from last to first. At x_i , collect the *bucket*: all constraints (original or previously recorded) whose scope contains x_i and no later variable. Join them, and *project out* x_i existentially: the recorded constraint holds of an assignment to the remaining scope variables iff *some* value of x_i extends it to satisfy the whole bucket. Place the recorded constraint in the bucket of its highest remaining variable and continue. This is adaptive consistency [6], later renamed and generalized as the constraint instance of *bucket elimination*, Dechter’s unifying dynamic-programming scheme that also captures directional resolution, Fourier elimination, and the elimination algorithms of probabilistic inference [4]. The textbook treatment is Dechter [5, ch. 4].

Three claims make adaptive consistency a *complete solver*, and a mechanization must separate them cleanly:

1. **Solution-set preservation.** Each recorded constraint is a *projection* of a join of existing constraints, hence implied by them: adding it changes nothing for full assignments, and restricting to the remaining variables loses exactly the eliminated coordinate. Formally, if S is the solution set of the current network over variables V and the step eliminates v , the new network’s solution set over V is $\exists_v S$ — the projection. In particular feasibility is preserved *exactly*, with no substituted values; this is where projection differs essentially from the conditioning (value-substitution) step, whose verdict is conditional on the chosen values.
2. **Enforcement of directional consistency.** After the run, every assignment to $x_0, \dots, x_{i-1}$ that satisfies all constraints among them — *including the recorded ones* — extends to x_i : the

recorded projection of bucket i is exactly the condition for a witness to exist, and it was placed among the earlier constraints. Note the logical form: the extension property is *relative* to the earlier constraints, not absolute; this distinction matters for the mechanization (Section 4.2).

3. **Cost.** Each bucket’s join and projection touch only the variables in that bucket’s scopes; if the ordering has induced width w (Section 3), every recorded scope has at most w variables, so each table has at most k^{w+1} rows over domains of size k , and the total work is $O(n k^{w+1})$ [5, 6].

Given (1)–(3), the backtrack-free construction of Freuder [8] (in its directional form) extracts a solution with no search, and the decision “is the network solvable?” is read off the final state. Dechter and Pearl also proved the ordering-graph identification that makes (3) sharp, and tree clustering — the join-tree compilation whose cluster sizes are governed by the same parameter — as the dual, query-oriented formulation [7].

3 The width formalisms

Three formalisms measure the same quantity.

Induced width (elimination width). For an ordering σ of the vertices of a graph G , process vertices from last to first, connecting the earlier neighbours of each processed vertex (the *fill-in*); the induced width $w^*(\sigma)$ is the maximal number of earlier neighbours in the filled graph, and the induced width of G is the minimum over orderings. This is precisely the maximal recorded-scope size of adaptive consistency along σ [5, 6], and it is the “dimension” of Bertelè and Brioschi’s nonserial dynamic programming, where the notion (under the name of that book) predates the constraint literature by fifteen years [2].

Partial k -trees. A k -tree is built from a $(k+1)$ -clique by repeatedly attaching a new vertex adjacent to a k -clique; partial k -trees are their subgraphs. Arnborg, Corneil and Proskurowski proved that recognizing partial k -trees — equivalently, deciding treewidth — is NP-hard when k is part of the input, and gave an $O(n^{k+2})$ algorithm for fixed k [1]. This is why every completeness theorem quantifies over a *given* ordering: finding the optimal one is itself intractable.

Tree decompositions. Robertson and Seymour introduced tree decompositions and treewidth in the Graph Minors series and showed the parameter’s algorithmic significance [12]. Bodlaender’s survey [3] assembles the equivalences: a graph has treewidth $\leq w$ iff it is a partial w -tree iff it has an elimination ordering of induced width $\leq w$ — the elimination ordering being read off a tree decomposition by eliminating leaves of bags, and conversely the fill-in cliques of an ordering forming the bags of a decomposition. It is exactly this last construction that `Ste.GraphTreewidth` mechanizes from the bucket-elimination side.

The circle closes with the converse direction: Grohe proved (for bounded arity, under a standard parameterized-complexity assumption) that bounded treewidth of the core is the *only* structural reason constraint classes are tractable [10] — adaptive consistency at bounded width is not just *a* tractable algorithm but essentially *the* tractable frontier.

4 The mechanization: what is proved, and where

4.1 Prior state of the repository

- `Ste.Elimination`: bucket elimination with a *substitutive* step (`bucketHead` joins the bucket of v and conditions on a chosen value $v \mapsto a$). Machine-proved: the fold faithfully computes

the elimination of the joint constraint (`joinConstraint_bucketEliminate`), a complete order decides the *substituted* instance (`bucketEliminate_decides`), and the total table space is $n k^{w+1}$ given bags of size $\leq w+1$ (`bucketEliminate_total_space`). The verdict is conditional on the substituted values — claim (1) of Section 2 fails for this step.

- **Ste.Projection**: the existential step `project` on monolithic constraints. Machine-proved: projection preserves nonemptiness exactly (`project_nonempty_iff`) and a complete projective run decides feasibility unconditionally (`projectEliminate_decides_feasibility`) — but with no bucket data structure, hence no width accounting.
- **Ste.Treedecomposition**: `AchievesWidth` and the attained minimum `inducedTreewidth`, discharging the free width hypothesis of the space bound.
- **Ste.GraphTreewidth**: tree decompositions from scratch, the elimination decomposition, and the headline `treewidth_primalGraph_le: tw(primal)  $\leq$  inducedTreewidth + 1`.
- **Ste.AdaptiveConsistency**: the bucket-network formalism (`sep i, R i, SepSupported, SepWidthLE`) and the sufficiency theorem `directionalConsistent_solvable: nonempty domains + directional consistency  $\Rightarrow$  a global solution, greedily`.

4.2 A gap surfaced by the enforcement analysis

`Ste.AdaptiveConsistency`’s `DirectionalConsistent` demands, for *every* domain-respecting assignment g , a witness $c \in D_{i+1}$ with $R_i(g[x_{i+1} \mapsto c])$. But what bucket elimination establishes at bucket i is weaker: a witness exists for assignments that satisfy the *earlier buckets* — because the recorded projection of bucket i is itself one of those earlier constraints. Dechter’s directional consistency is the relative notion [5, ch. 4]. Thus the repository’s sufficiency theorem, as previously stated, consumed a hypothesis strictly stronger than what the enforcement algorithm can supply: the two halves did not compose. This observation — easy to miss on paper, where “consistent partial assignment” silently carries the recorded constraints — is, we believe, the kind of precision dividend one mechanizes for.

4.3 The new module `Ste.BucketConsistency`

All statements below compile in CI with no `sorry`, `admit`, or added axioms; the run number is recorded in the accompanying log entry.

Part 1 (enforcement at the joint-constraint level). The projective bucket step `projectBucketStep` replaces the bucket of v by its joined *projection* (the recorded message is definitionally `project` of the joined bucket, `projectBucketHead_snd`). Machine-proved:

- `joinConstraint_projectBucketStep`: the residual state’s joint constraint *equals* $\exists_v$ (previous joint constraint) — claim (1), solution-set preservation, exactly;
- `exists_update_mem_of_mem_projectBucketStep`: every joint solution of the residual state extends at v alone to a joint solution of the previous state — claim (2) in instance form, the extension property the recorded projection buys;
- `joinConstraint_projectBucketEliminate, projectBucketEliminate_nonempty_iff, projectBucketEliminate`: the fold computes `projectEliminate`, preserves feasibility exactly, and a complete order yields the *unconditional* feasibility verdict — projective bucket elimination is a complete solver;

- `exists_extension_of_mem_projectBucketEliminate`: backtrack-free extraction — any survivor of the full run extends to a solution of the original instance changing only eliminated variables.

Part 2 (the relative sufficiency theorem). The gap of Section 4.2 is closed from the sufficiency side: `BucketRelativeConsistent` formalizes Dechter’s relative directional consistency, and `bucketRelativeConsistent_solvable(_from)` proves it implies global solvability. The proof is a *reduction*, not a new greedy induction: gate each bucket by its predecessors (`guardedR`), observe that the gated network has the same solution set (`adaptiveSolution_guardedR_iff`, by strong induction on the node index), is `SepSupported` for the fat prefix separators, and is *unconditionally* directionally consistent (an update at x_{i+1} is invisible to earlier buckets, so an unsatisfied guard stays unsatisfied), then invoke the existing engine. Solvability needs no width bound, so the fat separators are harmless.

Part 3 (the width link). Machine-proved: the projective and substitutive folds materialize identical scope traces (`projectBucketBags_mapfst` — scopes evolve by scope-only rules), so `Ste.Treewidth`’s width accounting transfers verbatim (`projectBucketBags_card_le`). Along the decreasing order on $\text{Fin}(n+1)$ — the order implicit in `Ste.AdaptiveConsistency`’s numbering — the recorded message scope at the step eliminating x_{i+1} defines `runSep`, with: `runSep_eq_messageScope` (it is literally the `bucketHead`-style scope of the reached state), `runSep_precedes` (topological order: the `hsep` hypothesis of the solvability theorems, now *established* by the run), and `runSep_widthLE` / `runSep_link`: if the decreasing order witnesses `AchievesWidth B w` then `SepWidthLE (runSep B) (w+1)` and `inducedTreewidth B ≤ w`. Via `treewidth_primalGraph_le`, the separator bound of the bucket-network formalism, the elimination width, and (up to the fixed offset) primal-graph treewidth are controlled by the same run.

4.4 Honest accounting of what remains open

1. **The per-node reassembly.** Part 1 preserves solutions and extracts them at the *joint-constraint* level; Part 2 proves the *per-node* relative condition sufficient. The missing link is the reassembly: packaging the final state of a run over $\text{Fin}(n+1)$ as a literal bucket network (D, runSep, R) with `adaptiveSet D R = joinConstraint B` and `BucketRelativeConsistent D R` — requiring constraint-to-bucket provenance (each constraint joins the bucket of its maximal variable) and root-domain extraction (surviving scope- $\{x_0\}$ constraints become D_0). On paper this is bookkeeping; in Lean it is a substantial indexing argument, and it is *not* mechanized. The completeness claim is therefore machine-proved in its joint form and paper-joined in its per-node form.
2. **General orders.** The width link is proved for the decreasing order; an arbitrary order is handled classically by renumbering the variables along it. The permutation transport is not mechanized.
3. **Time complexity.** Only table-*space* bounds exist in the repository (`Ste.Treewidth`, `Ste.Elimination`); running time is not modelled. The claim “cost exponential only in w ” is machine-proved for space, cited for time [5, 6].

5 Annotated bibliography

Each annotation is our own summary, written against the source’s role in this project.

- Freuder [8]** Defines the width of an ordered constraint graph as the maximal number of earlier neighbours and proves that strong $(w+1)$ -consistency plus width w yields backtrack-free search; trees with arc consistency are the $w=1$ case. For this project it is the ancestral form of the sufficiency half: `Ste.ConsistencyTree` mechanizes its tree case and `Ste.AdaptiveConsistency` its order-directional generalization. The paper’s own limitation — enforcing the required consistency can raise the width — is precisely what adaptive consistency later repairs, and why enforcement is the harder half to state correctly.
- Freuder [9]** Extends the 1982 theorem from backtrack-free to backtrack-bounded: b -level relative consistency along orderings whose width is controlled at level b bounds the backtrack depth. The analysis runs through k -trees, making explicit that Freuder’s width- w graphs are partial w -trees — the earliest bridge between the CSP width and the graph-minors treewidth formalisms, and the reason `Ste.GraphTreewidth`’s bridge theorem has classical warrant.
- Dechter and Pearl [6]** Introduces adaptive consistency — per-variable directional consistency enforced by recording the projected bucket constraint — together with the induced-width parameter w^* and the $O(n k^{w^*+1})$ bound, and proves the solution set is preserved because every recorded constraint is implied by the ones it is derived from. This is the primary source for the enforcement half attacked here: our `joinConstraint_projectBucketStep` is its preservation lemma made exact (equality with the projection, not merely implication), and our `BucketRelativeConsistent` is its output condition stated with the relativization made explicit.
- Dechter and Pearl [7]** Recasts the same machinery in compiled form: build a join tree whose clusters are the maximal cliques of a triangulation of the constraint graph, solve each cluster, and propagate. Cluster size is induced width plus one, so tree clustering and adaptive consistency have the same exponent; the join-tree view is the one `Ste.JunctionTree` and `Ste.Treedecomp` echo. Included because the equivalence of the per-order (elimination) and per-decomposition (cluster) presentations is exactly the width link mechanized in Part 3.
- Dechter [4]** Presents bucket elimination as a single elimination scheme instantiated by a “combine” and a “project” operator, with adaptive consistency the $\{\text{join}, \exists\}$ instance and directional resolution, Gaussian/Fourier elimination and probabilistic elimination as siblings. Its clean separation of the data structure (buckets, one pass) from the algebra (the two operators) is the design our `projectBucketHead/projectBucketStep` follows: the module swaps `Ste.Elimination`’s substitutive operator for the existential one and re-proves the fold correctness at that generality.
- Dechter [5]** The textbook consolidation (ch. 4 for adaptive consistency and induced width). Valuable here for its careful statement of *directional* consistency as extension of consistent partial assignments — consistent *with respect to all constraints among the earlier variables, recorded ones included* — which is the reading our `BucketRelativeConsistent` formalizes and which the repository’s earlier `DirectionalConsistent` unintentionally strengthened (Section 4.2).
- Mackworth [11]** The systematization of node, arc and path consistency and their propagation algorithms as constraint-network fixed points. Included as the baseline the width theory refines: uniform local consistency neither implies solvability nor respects structure, and the entire adaptive line can be read as replacing Mackworth’s uniform closure with an order-adapted one.

- Bertelè and Brioschi [2]** The nonserial dynamic programming monograph: eliminating variables from an objective decomposed over a hypergraph, with cost governed by what is now called induced width (their “dimension”), including the NP-hardness flavour of choosing the elimination order. Adaptive consistency is its Boolean (feasibility) specialization; we cite it as the origin of the elimination-width parameter that `Ste.TreeDecomp`’s `inducedTreewidth` mechanizes.
- Seidel [13]** The invasion procedure: sweep the variables once, maintaining the set of partial solutions restricted to the current frontier; time exponential in the maximal frontier. Historically the first CSP decision procedure exponential only in a separator-like parameter, and conceptually the ancestor of our Part 1: the joint-constraint-level fold with exact feasibility preservation is an invasion-style argument, with projections replacing frontier tables.
- Arnborg et al. [1]** Proves NP-completeness of deciding treewidth (partial k -tree recognition) for variable k and gives the $O(n^{k+2})$ algorithm for fixed k . For this project it justifies the shape of every mechanized statement: all width bounds are stated for a *given* order or decomposition, and `inducedTreewidth` is defined as an attained infimum rather than computed.
- Robertson and Seymour [12]** Introduces tree decompositions and treewidth and initiates their algorithmic use in the Graph Minors project. The three axioms of `Ste.GraphTreewidth`’s `TreeDecomposition` (vertex cover, edge cover, running intersection in rooted form) are this paper’s definition, specialized to a list-of-bags presentation suitable for Lean induction.
- Bodlaender [3]** The survey establishing the equivalence web around treewidth: partial k -trees, elimination orderings, cops-and-robber games, brambles. The elimination-order-to-decomposition direction surveyed there is the one `Ste.GraphTreewidth` mechanizes (`treewidth_primalGraph_le`); the converse direction (decomposition to ordering, needed to replace “the decreasing order” by “some optimal order” in Part 3 without renumbering) is surveyed there too and remains unmechanized.
- Grohe [10]** The converse: for bounded-arity classes, tractability of the homomorphism/CSP problem forces bounded treewidth of the cores (assuming $\text{FPT} \neq \text{W}[1]$). Cited to position the mechanized material: the enforcement theorems formalize the *algorithmic* side of a boundary that is, by this result, information-theoretically tight.

References

- [1] Stefan Arnborg, Derek G. Corneil, and Andrzej Proskurowski. Complexity of finding embeddings in a k -tree. *SIAM Journal on Algebraic and Discrete Methods*, 8(2):277–284, 1987. doi: 10.1137/0608024.
- [2] Umberto Bertelè and Francesco Brioschi. *Nonserial Dynamic Programming*, volume 91 of *Mathematics in Science and Engineering*. Academic Press, New York, 1972.
- [3] Hans L. Bodlaender. A partial k -arboretum of graphs with bounded treewidth. *Theoretical Computer Science*, 209(1–2):1–45, 1998. doi: 10.1016/S0304-3975(97)00228-4.
- [4] Rina Dechter. Bucket elimination: A unifying framework for reasoning. *Artificial Intelligence*, 113(1–2):41–85, 1999. doi: 10.1016/S0004-3702(99)00059-4.
- [5] Rina Dechter. *Constraint Processing*. Morgan Kaufmann, San Francisco, 2003.

- [6] Rina Dechter and Judea Pearl. Network-based heuristics for constraint-satisfaction problems. *Artificial Intelligence*, 34(1):1–38, 1987. doi: 10.1016/0004-3702(87)90002-6.
- [7] Rina Dechter and Judea Pearl. Tree clustering for constraint networks. *Artificial Intelligence*, 38(3):353–366, 1989. doi: 10.1016/0004-3702(89)90037-4.
- [8] Eugene C. Freuder. A sufficient condition for backtrack-free search. *Journal of the ACM*, 29(1):24–32, 1982. doi: 10.1145/322290.322292.
- [9] Eugene C. Freuder. A sufficient condition for backtrack-bounded search. *Journal of the ACM*, 32(4):755–761, 1985. doi: 10.1145/4221.4225.
- [10] Martin Grohe. The complexity of homomorphism and constraint satisfaction problems seen from the other side. *Journal of the ACM*, 54(1):1–24, 2007. doi: 10.1145/1206035.1206036.
- [11] Alan K. Mackworth. Consistency in networks of relations. *Artificial Intelligence*, 8(1):99–118, 1977. doi: 10.1016/0004-3702(77)90007-8.
- [12] Neil Robertson and Paul D. Seymour. Graph minors. II. algorithmic aspects of tree-width. *Journal of Algorithms*, 7(3):309–322, 1986. doi: 10.1016/0196-6774(86)90023-4.
- [13] Raimund Seidel. A new method for solving constraint satisfaction problems. In *Proceedings of the 7th International Joint Conference on Artificial Intelligence (IJCAI-81)*, pages 338–342, Vancouver, 1981.